

```

//-----
//
// Name:      vadd_cpp.cpp
//
// Purpose:   Elementwise addition of two vectors (c = a + b)
//
//              c = a + b
//
// HISTORY:   Written by Tim Mattson, June 2011
//            Ported to C++ Wrapper API by Benedict Gaster, September 2011
//            Updated to C++ Wrapper API v1.2 by Tom Deakin and Simon McIntosh-Smit
h, October 2012
//            Updated to C++ Wrapper v1.2.6 by Tom Deakin, August 2013
//
//-----

#define __CL_ENABLE_EXCEPTIONS

#include "cl.hpp"

#include "util.hpp" // utility library

#include "err_code.h"

#include <vector>
#include <cstdio>
#include <cstdlib>
#include <string>

#include <iostream>
#include <fstream>

// pick up device type from compiler command line or from the default type
#ifndef DEVICE
#define DEVICE CL_DEVICE_TYPE_DEFAULT
#endif

//-----

#define TOL      (0.001)    // tolerance used in floating point comparisons
#define LENGTH  (1024)     // length of vectors a, b, and c

int main(void)
{
    std::vector<float> h_a(LENGTH);           // a vector
    std::vector<float> h_b(LENGTH);           // b vector
    std::vector<float> h_c(LENGTH, 0xdeadbeef); // c = a + b, from compute device

    cl::Buffer d_a;                          // device memory used for the input a vector
    cl::Buffer d_b;                          // device memory used for the input b vector
    cl::Buffer d_c;                          // device memory used for the output c vector

    // Fill vectors a and b with random float values
    int count = LENGTH;
    for(int i = 0; i < count; i++)
    {
        h_a[i] = rand() / (float)RAND_MAX;
        h_b[i] = rand() / (float)RAND_MAX;
    }

    try
    {
        // Create a context
        cl::Context context(DEVICE);

        // Load in kernel source, creating a program object for the context
        cl::Program program(context, util::loadProgram("vadd.cl"), true);
    }
}

```

```

// Get the command queue
cl::CommandQueue queue(context);

// Create the kernel functor
auto vadd = cl::make_kernel<cl::Buffer, cl::Buffer, cl::Buffer, int>(program
, "vadd");

d_a  = cl::Buffer(context, begin(h_a), end(h_a), true);
d_b  = cl::Buffer(context, begin(h_b), end(h_b), true);

d_c  = cl::Buffer(context, CL_MEM_WRITE_ONLY, sizeof(float) * LENGTH);

util::Timer timer;

vadd(
    cl::EnqueueArgs(
        queue,
        cl::NDRange(count)),
    d_a,
    d_b,
    d_c,
    count);

queue.finish();

double rtime = static_cast<double>(timer.getTimeMilliseconds()) / 1000.0;
printf("\nThe kernels ran in %lf seconds\n", rtime);

cl::copy(queue, d_c, begin(h_c), end(h_c));

// Test the results
int correct = 0;
float tmp;
for(int i = 0; i < count; i++) {
    tmp = h_a[i] + h_b[i]; // expected value for d_c[i]
    tmp -= h_c[i];         // compute errors
    if(tmp*tmp < TOL*TOL) { // correct if square deviation is less
        correct++;         // than tolerance squared
    }
    else {
        printf(
            " tmp %f h_a %f h_b %f h_c %f \n",
            tmp,
            h_a[i],
            h_b[i],
            h_c[i]);
    }
}

// summarize results
printf(
    "vector add to find C = A+B:  %d out of %d results were correct.\n",
    correct,
    count);
}

catch (cl::Error err) {
    std::cout << "Exception\n";
    std::cerr
        << "ERROR: "
        << err.what()
        << "("
        << err_code(err.err())
        << ")" "
        << std::endl;
}
}

```